

## 5.1 WIFI Connect ESP32

### ESP32 WiFi Connection

Note: This tutorial uses the ESP32 development board to connect to a WiFi router, which is the first step in learning ESP32 wireless communication. Please make sure your ESP32 has the correct drivers and board core support installed.

#### Related Knowledge

##### What is WiFi?

**WiFi** is a wireless local area network technology that allows electronic devices to connect to a network via radio waves. It is everywhere in daily life—phones, computers, and smart home devices all rely on WiFi to access the internet. This project simply makes the ESP32 board "connect to your home WiFi" just like a smartphone does.

**Simple analogy:** WiFi is like an "invisible web," the router is the "center of the spider web," and the ESP32 is a "little insect"—once it gets on the web, it can communicate with other insects (computers, phones).

##### STA Mode vs AP Mode

ESP32's WiFi has two working modes. Beginners need to understand the difference:

| Mode   | Full Name                 | Analogy                    | Description                                                        |
|--------|---------------------------|----------------------------|--------------------------------------------------------------------|
| STA    | Station                   | Phone connecting to router | ESP32 acts as a "client" connecting to an existing WiFi hotspot    |
| AP     | Access Point              | The router itself          | ESP32 creates its own WiFi hotspot for other devices to connect to |
| AP+STA | Both modes simultaneously | Both a router and a client | Can connect to others and be connected to by others                |



This tutorial uses **STA mode**, where the ESP32 connects to your home or office WiFi router.

#### ESP32 Development Board

**ESP32** is an IoT development board developed by Espressif:

- ▶ Built-in WiFi and Bluetooth, making it a mainstream choice for IoT development
- ▶ Dual-core processor, 240MHz main frequency, far outperforming the traditional Arduino UNO
- ▶ Supports STA/AP mode switching, flexibly adapting to various network scenarios
- ▶ In this project, the ESP32 acts as a "client" connecting to a specified WiFi

#### Four Stages of WiFi Connection

The process of connecting an ESP32 to a WiFi router consists of four steps:



- 1. Set Mode:** Tell the ESP32 to work in STA mode
- 2. Begin Connection:** Send a connection request to the router (requires SSID and password)
- 3. Wait for Negotiation:** Wait for the router to respond, retrying up to N times
- 4. Get Information:** After a successful connection, read the IP address, signal strength, and other info

#### MAC Address and IP Address

- ▶ **MAC Address:** The device's "physical ID card," globally unique (e.g., `24:6F:28:A4:B2:C1` ), determined by hardware
- ▶ **IP Address:** The device's "house number" in the network (e.g., `192.168.1.105` ), automatically assigned by the router
- ▶ **RSSI:** Signal strength indicator, measured in dBm. Values closer to 0 mean stronger signal (e.g., `-50` is better than `-80` )

#### Auto-Reconnect on Disconnect

When the WiFi signal is unstable, the device may disconnect. The **auto-reconnect** mechanism allows the device to automatically attempt to reconnect after a disconnection:

- Uses `millis()` for timing, checking the connection status at fixed intervals
- Calls `WiFi.reconnect()` when a disconnection is detected
- Non-blocking design, so the main loop can continue executing other tasks

[Back to Elegoo](#)

**Simple analogy:** Just like a phone automatically reconnects to WiFi after a disconnection, without requiring manual operation.

## Library Installation Guide

This project uses the `WiFi` library built into the ESP32 board. It is automatically available after installing the ESP32 core.

### Installing ESP32 Board Support

1. Open the Arduino IDE
2. Click **File** → **Preferences**
3. In "Additional Board Manager URLs," enter:

```
1 | https://dl.espressif.com/dl/esp32/esp32-arduino-index.json
```

4. Click **Tools** → **Board** → **Board Manager**
5. Search for `esp32` and install **esp32 by Espressif Systems**
6. After installation, select your ESP32 model (e.g., "ESP32 Dev Module") under **Tools** → **Board**

## Key Functions

### WiFi Library Core Functions

| Function                                | Description              | Role in This Project                                                                  |
|-----------------------------------------|--------------------------|---------------------------------------------------------------------------------------|
| <code>WiFi.mode(mode)</code>            | Set WiFi working mode    | Set to <code>WIFI_STA</code> station mode                                             |
| <code>WiFi.begin(ssid, password)</code> | Initiate WiFi connection | Connect to the router using the specified SSID and password                           |
| <code>WiFi.status()</code>              | Get current WiFi status  | Determine whether the connection succeeded (returns <code>WL_CONNECTED</code> , etc.) |
| <code>WiFi.localIP()</code>             | Get local IP address     | Print the IP obtained by the ESP32                                                    |
| <code>WiFi.macAddress()</code>          | Get MAC address          | Print the ESP32's physical address                                                    |
| <code>WiFi.RSSI()</code>                | Get signal strength      | Print the current WiFi signal strength (dBm)                                          |
| <code>WiFi.reconnect()</code>           | Attempt to reconnect     | Automatically reconnect when disconnected                                             |

### WiFi Status Constants

| Constant                        | Value | Description                   |
|---------------------------------|-------|-------------------------------|
| <code>WL_IDLE_STATUS</code>     | 0     | Idle status, not connected    |
| <code>WL_NO_SSID_AVAIL</code>   | 1     | Specified SSID not found      |
| <code>WL_SCAN_COMPLETED</code>  | 2     | Scan completed                |
| <code>WL_CONNECTED</code>       | 3     | <b>Successfully connected</b> |
| <code>WL_CONNECT_FAILED</code>  | 4     | Connection failed             |
| <code>WL_CONNECTION_LOST</code> | 5     | Connection lost               |
| <code>WL_DISCONNECTED</code>    | 6     | Disconnected                  |



This project mainly checks `WiFi.status() == WL_CONNECTED` to confirm the connection status.

### Time-Related Functions

| Function               | Description             | Role in This Project                                                      | <a href="#">Back to Elegoo</a> |
|------------------------|-------------------------|---------------------------------------------------------------------------|--------------------------------|
| <code>millis()</code>  | Milliseconds since boot | Calculate reconnect interval, implement non-blocking timing               |                                |
| <code>delay(ms)</code> | Blocking delay          | Pause for 500ms during connection wait, giving the router time to respond |                                |

# Complete Code Analysis

## 1. Header File and Variable Definitions

```
1 | #include <WiFi.h>
2 |
3 | //===== WiFi Credentials =====
4 | const char* ssid  = "ZNP"; // Replace with your WiFi name
5 | const char* password = "16881688"; // Replace with your WiFi password
6 |
7 | //===== Variables =====
8 | unsigned long previousMillis = 0;
9 | const long interval = 5000; // Reconnect check interval (ms)
```

### Code Explanation:

- `#include <WiFi.h>` : Includes the ESP32 WiFi library, providing all WiFi-related functions
- `ssid` : WiFi name (Service Set Identifier), the "name" of the router—this is what you see when connecting your phone to WiFi
- `password` : WiFi password, the router's encryption passphrase
- `previousMillis` : Records the timestamp of the last reconnect check
- `interval` : Reconnect check interval (5000ms = 5 seconds); the connection status is checked every this often



You need to change `ssid` and `password` to your own WiFi name and password.

## 2. `setup()` Initialization Function

`setup()` runs only once after the ESP32 powers on, and is responsible for completing all WiFi connection initialization.

### 2.1 Serial Initialization

```
1 | Serial.begin(115200);
2 | delay(1000);
3 |
4 | Serial.println("\n=====");
5 | Serial.println("ESP32 WiFi Tutorial");
6 | Serial.println("=====");
```

### Step Analysis:

- `Serial.begin(115200)` : Initializes the serial port with a baud rate of 115200, used to send debug information to the computer
- `delay(1000)` : Waits 1 second for the serial port to stabilize, avoiding missing early output
- `Serial.println()` : Prints a welcome banner to confirm the program is running normally

### 2.2 Set WiFi Mode

```
1 | WiFi.mode(WIFI_STA);
2 | Serial.println("WiFi mode set to STA (Station)");
```

### Code Analysis:

- `WiFi.mode(WIFI_STA)` : Sets the ESP32 to STA (Station) mode
- Only in STA mode can the ESP32 act as a "client" connecting to a router
- This function must be called before `WiFi.begin()`

### 2.3 Initiate WiFi Connection

```

1 | Serial.print("Connecting to ");
2 | Serial.println(ssid);
3 |
4 | WiFi.begin(ssid, password);

```

[Back to Elegoo](#)

#### Code Analysis:

- `WiFi.begin(ssid, password)` : Sends a connection request to the specified WiFi router
- After receiving the request, the router verifies the password and assigns an IP address
- This function "initiates a request" rather than "waits for a result," so it does not block

### 2.4 Wait for Connection Result

```

1 | int attempts = 0;
2 | while (WiFi.status() != WL_CONNECTED && attempts < 20){
3 |     delay(500);
4 |     Serial.print(".");
5 |     attempts++;
6 | }

```

#### Code Analysis:

This is a **blocking wait loop** with the following logic:

- `WiFi.status()` : Queries the current connection status in real time
- `!= WL_CONNECTED` : As long as it's not connected, keep waiting
- `attempts < 20` : Wait at most 20 times, 500ms each time, totaling about 10 seconds
- `delay(500)` : Pause for 500ms before checking again, giving the router time to process
- `Serial.print(".");` : Prints a dot in the serial monitor so you can see the waiting progress



Connecting a phone to WiFi also takes a few seconds, and the same is true for the ESP32. If the password is wrong or the signal is too weak, it will give up after 20 attempts.

### 2.5 Print Connection Information

```

1 | if (WiFi.status() == WL_CONNECTED) {
2 |     Serial.println("\n✅ WiFi Connected!");
3 |     Serial.print("IP Address: ");
4 |     Serial.println(WiFi.localIP());    // e.g. 192.168.1.105
5 |     Serial.print("MAC Address: ");
6 |     Serial.println(WiFi.macAddress()); // e.g. 24:6F:28:A4:B2:C1
7 |     Serial.print("RSSI (signal strength): ");
8 |     Serial.print(WiFi.RSSI());
9 |     Serial.println(" dBm");
10 | } else {
11 |     Serial.println("\n❌ WiFi Connection Failed!");
12 |     Serial.println("Please check your SSID and password.");
13 | }

```

#### Code Analysis:

- When **connection succeeds**, it prints three key pieces of information:
  - `WiFi.localIP()` : The IP address assigned to the ESP32 by the router (e.g., `192.168.1.105`), which is needed for subsequent HTTP/WebSocket communication
  - `WiFi.macAddress()` : The hardware address of the ESP32, used for network device identification
  - `WiFi.RSSI()` : Signal strength (in dBm); values closer to 0 are better (`-50` is strong, `-80` is weak)
- When **connection fails**, it prints an error message reminding you to check the SSID and password

## 3. `loop()` Main Loop Function

`loop()` is called repeatedly after `setup()` finishes. It is the "heartbeat" of the program.

#### Auto-Reconnect Logic

```

1 | void loop() {
2 |   // ----- Auto-reconnect logic -----
3 |   if (WiFi.status() != WL_CONNECTED) {
4 |     unsigned long currentMillis = millis();
5 |     if (currentMillis - previousMillis >= interval) {
6 |       previousMillis = currentMillis;
7 |       Serial.println("WiFi lost. Attempting to reconnect...");
8 |       WiFi.reconnect();
9 |     }
10 |   }
11 |
12 |   // Your main application code goes here
13 |   // e.g. HTTP client, WebSocket, MQTT, etc.
14 |
15 |   delay(100);
16 | }

```

### Step-by-Step Analysis of Core Logic:

#### Step 1: Detect Disconnection

```

1 | if (WiFi.status() != WL_CONNECTED) {

```

Each loop checks the WiFi status. If it's not equal to `WL_CONNECTED`, it means the connection has been lost or was never established.

#### Step 2: Non-Blocking Timing

```

1 | unsigned long currentMillis = millis();
2 | if (currentMillis - previousMillis >= interval) {
3 |   previousMillis = currentMillis;

```

Uses `millis()` to implement non-blocking timing:

- `millis()` returns the number of milliseconds since the ESP32 booted (overflows after about 50 days)
- `currentMillis - previousMillis` calculates how long it has been since the last check
- Only reconnect if more than `interval` (5 seconds) has passed, avoiding excessive frequency



Why use `millis()` instead of `delay()`? Because `delay()` blocks the entire program, while with `millis()` the main loop can continue executing other tasks during the timing period.

#### Step 3: Perform Reconnection

```

1 | Serial.println("WiFi lost. Attempting to reconnect...");
2 | WiFi.reconnect();

```

- Prints a reconnect message
- `WiFi.reconnect()` : Tells the ESP32 to send a new connection request to the router
- `previousMillis = currentMillis` : Resets the timer to start timing for the next reconnect attempt

#### Step 4: Short Delay

```

1 | delay(100);

```

Pauses for 100ms each loop, giving the hardware and network time to process and avoiding CPU overload.

## Operation Steps Guide

### Step 1: Modify WiFi Credentials

Open `wifi_connect.ino` and modify these two lines:

```
1 | const char* ssid = "YourWiFiName";
2 | const char* password = "YourWiFiPassword";
```

[Back to Elegoo](#)



**Note:** ESP32 only supports 2.4GHz WiFi, not 5GHz. Please make sure your router has the 2.4G band enabled.

## Step 2: Connect Hardware

1. Connect the ESP32 development board to your computer using a USB data cable
2. Make sure the cable supports data transfer (some cables are for charging only)

## Step 3: Upload the Program

1. Open the Arduino IDE
2. Select the board: **Tools → Board → ESP32 Dev Module** (or your specific model)
3. Select the port: **Tools → Port → the correct COM port**
4. Click the **Upload** button (→ arrow icon)
5. Wait for compiling and uploading to complete (may take a few minutes the first time)

## Step 4: View Output

1. Open the Serial Monitor (magnifying glass icon in the toolbar)
2. Set the baud rate to **115200** (bottom right corner)
3. Observe the serial output to confirm the WiFi connection status

## Expected Serial Output

### When Connection Succeeds

```
1 | =====
2 | ESP32 WiFi Tutorial
3 | =====
4 | WiFi mode set to STA (Station)
5 | Connecting to ZNP_2.4G
6 | .....
7 | ✅ WiFi Connected!
8 | IP Address: 192.168.1.105
9 | MAC Address: 24:6F:28:A4:B2:C1
10 | RSSI (signal strength): -45 dBm
```

### When Connection Fails

```
1 | =====
2 | ESP32 WiFi Tutorial
3 | =====
4 | WiFi mode set to STA (Station)
5 | Connecting to MyWiFi
6 | .....
7 | ❌ WiFi Connection Failed!
8 | Please check your SSID and password.
```

### When Reconnecting After a Disconnect

```
1 | WiFi lost. Attempting to reconnect...
2 | WiFi lost. Attempting to reconnect...
3 | .....
4 | ✅ WiFi Connected!
5 | IP Address: 192.168.1.105
6 | ...
```

## FAQ

### Common Questions

#### Q1: The Serial Monitor shows garbled characters?

A: Please make sure the baud rate of the Serial Monitor is set to **115200** (bottom right dropdown).

#### Q2: The serial shows "WiFi Connection Failed"?

A: Please check the following in order:

- Is the SSID and password correct? (case-sensitive, watch for spaces)
- Is the router on the 2.4GHz band? (ESP32 does not support 5GHz)
- Is the ESP32 too far from the router? (within 5 meters is recommended)
- Has the router enabled MAC address filtering?

**Q3: The serial only prints a few dots and then stops?**

A: The board model may be selected incorrectly. Please confirm you have selected the correct ESP32 model (e.g., ESP32 Dev Module, ESP32S3 Dev Module, etc.).

**Q4: The RSSI value is -80 or lower?**

A: The signal is weak. It is recommended to shorten the distance between the ESP32 and the router, or use an ESP32 module with an external antenna.

**Q5: No output at all after uploading the program?**

A: Please check:

- Does the USB cable support data transfer? (try another cable)
- Is the Serial Monitor open?
- Is the baud rate 115200?

**Q6: WiFi disconnects shortly after connecting?**

A: Possible causes:

- The router signal is unstable; try changing the channel
- The ESP32 power supply is insufficient; independent 5V power is recommended
- The router has reached its device limit (some routers have a maximum number of connected devices)

**Troubleshooting Table**

| Problem                 | Possible Cause              | Solution                                     |
|-------------------------|-----------------------------|----------------------------------------------|
| Garbled serial output   | Wrong baud rate             | Change to 115200                             |
| Cannot upload           | Wrong port / missing driver | Check port and CH340/CP2102 driver           |
| WiFi won't connect      | Wrong SSID/password         | Double-check case and spaces                 |
| WiFi won't connect      | 5GHz band                   | Use 2.4GHz                                   |
| Weak signal             | Too far / obstruction       | Move closer to router or use an antenna      |
| Frequent disconnections | Insufficient power          | Use an independent 5V power supply           |
| Reconnect too slow      | Interval too long           | Modify the <a href="#">interval</a> variable |

**Modification Suggestions**

**1. Adjust the Reconnect Interval**

Modify the [interval](#) variable in the code:

```
1 | const long interval = 3000; // Change to 3 seconds, more frequent reconnection
2 | const long interval = 10000; // Change to 10 seconds, fewer checks
```

**2. Static IP Address (Optional)**

If you need a fixed IP instead of dynamic allocation, add the following before `WiFi.begin()` :

```
1 | IPAddress ip(192, 168, 1, 150); // Target static IP
2 | IPAddress gateway(192, 168, 1, 1); // Router gateway
3 | IPAddress subnet(255, 255, 255, 0); // Subnet mask
4 | WiFi.config(ip, gateway, subnet);
```

**3. Add WiFi Scanning**

```

1 | WiFi.mode(WIFI_STA);
2 | int n = WiFi.scanNetworks();
3 | Serial.print("Found ");
4 | Serial.print(n);
5 | Serial.println(" networks:");
6 | for (int i = 0; i < n; i++) {
7 |   Serial.print(i + 1);
8 |   Serial.print(": ");
9 |   Serial.print(WiFi.SSID(i));
10 |  Serial.print(" (");
11 |   Serial.print(WiFi.RSSI(i));
12 |   Serial.println(" dBm)");
13 | }

```

#### 4. Add Business Logic After Connection

Add actual functionality at the commented location in `loop()` :

```

1 | void loop() {
2 |   if (WiFi.status() != WL_CONNECTED) {
3 |     // ...reconnect logic ...
4 |   }
5 |
6 |   // Add your application code here
7 |   // e.g., send a heartbeat to the server every 30 seconds
8 |   static unsigned long lastSend = 0;
9 |   if (millis() - lastSend > 30000) {
10 |    lastSend = millis();
11 |    Serial.println("Sending heartbeat...");
12 |    // HTTP client, MQTT publish, etc.
13 |   }
14 |
15 |   delay(100);
16 | }

```

#### 5. Switch to AP Mode

If you want the ESP32 to create its own hotspot for other devices to connect to (instead of connecting to a router):

```

1 | // AP mode example
2 | const char* apSSID = "ESP32_AP";
3 | const char* apPassword = "12345678";
4 |
5 | void setup() {
6 |   WiFi.mode(WIFI_AP);
7 |   WiFi.softAP(apSSID, apPassword);
8 |   IPAddress IP = WiFi.softAPIP();
9 |   Serial.print("AP IP address: ");
10 |   Serial.println(IP);
11 | }

```

## Notes

### Note 1: 2.4GHz Limit

ESP32 only supports **2.4GHz WiFi**, not 5GHz. If your router only has the 5GHz band enabled, the ESP32 will not be able to connect. Please enable 2.4GHz in your router settings.

### Note 2: Password Length Limit

The WiFi password must be at least 8 characters long. If your router's password is too short, please change the router password or use a different network.

### Note 3: Power Supply Requirements

The ESP32's WiFi function consumes a lot of power (peak can reach 300mA). It is recommended to use an independent 5V power supply through the VIN pin, rather than relying solely on USB.

### Note 4: Non-Blocking Reconnect

The auto-reconnect logic uses `millis()` for a non-blocking design, but `WiFi.reconnect()` itself is blocking (about 1 second). In practical applications, if high real-time performance is required, you can use `WiFi.begin()` to re-initiate the connection instead.

**Note 5: Public WiFi**

[Back to Elegoo](#)

Connecting to public WiFi (such as cafes, campus networks) may require web authentication (Portal), which the ESP32 cannot complete. Please use an open or WPA2-encrypted network.

**Note 6: Special Characters in SSID**

When the SSID contains Chinese or special characters, encoding issues may occur. It is recommended to use an SSID with only English letters and numbers.

[← Overview](#)

[AP mode→](#)